

PRD - API Reliability & Testing Platform

1. Product Overview

Working Name

TBD

Product

A continuous API reliability testing platform that verifies whether APIs are functioning correctly, detects regressions, and alerts developers when expected API behavior changes.

Positioning

Don't just monitor whether your API is up. Verify that it's working correctly.

Core Value Proposition

The platform continuously tests API endpoints against developer-defined expectations and detects:

- Downtime
 - Unexpected HTTP responses
 - Slow response times
 - Response/body changes
 - Contract violations
 - Authentication failures
 - Other defined behavioral regressions
-

2. Problem Statement

Most basic API monitoring focuses on availability.

For example:

```
GET /api/orders  
→ 200 OK  
→ Healthy
```

But a 200 OK response does not necessarily mean the API is functioning correctly.

The API could:

- Return incorrect data
- Remove required fields
- Change a field's data type
- Return empty data
- Become significantly slower

- Change its response structure
- Break authentication
- Behave differently after deployment

The platform should detect these problems automatically.

3. Target Users

Primary Users

Individual Developers

Developers building:

- SaaS applications
- REST APIs
- Mobile backends
- Internal APIs
- Third-party integrations

Small Engineering Teams

Teams that need API reliability testing without the complexity of enterprise monitoring platforms.

Agencies

Agencies responsible for maintaining APIs for multiple clients.

Secondary Users

- QA engineers
 - DevOps engineers
 - Technical leads
-

4. MVP Goals

The MVP must allow a user to:

1. Create an account.
2. Add an API endpoint.
3. Configure expected behavior.
4. Execute the test.
5. Schedule recurring tests.
6. Store test results.
7. Detect failures and regressions.

8. View detailed failure information.
 9. Receive alerts.
-

5. Core User Flow

```
Create Account
  ↓
Create Project
  ↓
Add API Monitor
  ↓
Configure Request
  ↓
Define Assertions
  ↓
Run Test
  ↓
View Result
  ↓
Schedule Monitor
  ↓
Automatic Execution
  ↓
Failure Detection
  ↓
Notification
```

6. MVP Features

6.1 Authentication

Users can:

- Register
- Login
- Logout
- Reset password
- Manage basic account information

MVP

Email/password authentication.

Social login can come later.

6.2 Projects

Users can create projects to organize monitors.

Example:

Projects

My SaaS API
Client A API
Payment API

Each project contains multiple monitors.

6.3 API Monitor Creation

A user can create a monitor.

Request configuration

Support:

- GET
- POST
- PUT
- PATCH
- DELETE

Configuration:

Name
URL
HTTP method
Headers
Query parameters
Request body
Authentication
Timeout

6.4 Authentication

MVP should support:

- No authentication
- API key
- Bearer token
- Basic authentication

Secrets must never be displayed in plaintext after being saved.

6.5 Assertions

Assertions define what the platform expects from the API.

HTTP Assertions

- Status code equals
- Status code range
- Response time less than X ms

Header Assertions

- Header exists
- Header equals value

JSON Assertions

- Field exists
- Field does not exist
- Field type
- Field equals value
- Field contains value

Example:

```
Status = 200
```

```
Response time < 500ms
```

```
$.data.id exists
```

```
$.data.email type = string
```

```
$.data.status = "active"
```

6.6 Test Execution

When a test runs, the system records:

```
Request  
Response  
Status code  
Response time  
Assertions  
Passed assertions  
Failed assertions  
Timestamp  
Error
```

6.7 Scheduling

Users can configure:

- Every 1 minute
- Every 5 minutes
- Every 15 minutes
- Every 30 minutes
- Every hour
- Every 6 hours
- Every 24 hours

The backend scheduler should trigger jobs rather than keeping long-running timers inside the web application.

6.8 Failure Detection

The system should detect:

Availability Failure

Connection refused
Timeout
DNS failure
TLS failure

HTTP Failure

Expected: 200
Received: 500

Performance Failure

Expected: <500ms
Received: 1,842ms

Contract Failure

Expected:
\$.user.email → string

Received:
\$.user.email → null

6.9 Incident Creation

Repeated failures should create an incident.

Example:

API INCIDENT

GET /api/users

Status: Failing

First detected:
14:32

Reason:
Expected status 200
Received status 500

The system should avoid creating a new incident for every failed execution.

One incident can contain many failed test runs.

6.10 Recovery Detection

When a failing monitor becomes healthy again:

INCIDENT RESOLVED

GET /api/users

Downtime:
8 minutes 42 seconds

Failed checks:
14

6.11 Notifications

MVP:

- Email
- Webhook

Notifications should support:

Failure

API monitor failed

Recovery

API monitor recovered

Users should eventually be able to control notification preferences.

6.12 Dashboard

The dashboard should show:

Overall Health

12 Monitors
10 Healthy
1 Degraded
1 Failing

Also:

- Recent incidents
 - Recent test results
 - Average response time
 - Monitor status
 - Uptime percentage
-

6.13 Monitor Detail

Each monitor should provide:

Overview

- Current status
- Uptime
- Response time
- Last check
- Next check

Test history

Time	Status	Response
14:00	Passed	182ms
14:05	Passed	194ms
14:10	Failed	500
14:15	Failed	500
14:20	Passed	201ms

Failure details

Show:

- Request
 - Response
 - Expected
 - Actual
 - Failed assertion
 - Error message
-

7. MVP Non-Functional Requirements

Performance

Normal dashboard requests should respond quickly.

API test execution should have configurable timeouts.

Reliability

A failed monitored API must not cause the monitoring system itself to fail.

Workers should handle failed jobs safely.

Security

The system must protect:

- API credentials
- Authentication tokens
- User data
- Request bodies
- Response data

Scalability

The architecture should allow additional workers to be added as monitoring volume increases.

8. Security Requirements

This is particularly important because the platform will make outbound HTTP requests based on user input.

SSRF protection

The monitoring engine must prevent users from using the platform to access:

localhost
127.0.0.1
Private IP ranges
Cloud metadata endpoints
Internal services

This is a **critical requirement**, not a future feature.

Secrets

API keys and tokens should be encrypted at rest.

They should never appear in:

- Logs
- Error messages
- Dashboard responses
- Notifications

Rate limiting

Protect:

- Authentication endpoints
 - Monitor creation
 - Test execution
 - API endpoints
-

9. MVP Out of Scope

Do not build these initially:

- Load testing
 - Full API client
 - API documentation
 - Kubernetes monitoring
 - Server monitoring
 - Database monitoring
 - AI assistant
 - Automatic contract generation
 - GitHub deployment integration
 - Slack/Discord integrations
 - Mobile application
 - SSO
 - Enterprise RBAC
 - Billing
-

10. Future Features

Potential V2/V3 capabilities:

Contract Discovery

Automatically observe responses and suggest API contracts.

Deployment Detection

Connect GitHub/GitLab and correlate failures with deployments.

Regression Analysis

Identify performance and behavioral changes.

AI Failure Analysis

Explain likely causes using test history and deployment information.

Advanced Workflows

```
Login
↓
Get token
↓
Create order
↓
Retrieve order
↓
Verify response
```

Multi-region Testing

Run the same test from different geographic locations.

11. MVP Success Criteria

The MVP is successful when a developer can:

Add an API → define expected behavior → schedule tests → let the system continuously test it → receive an alert when behavior breaks → inspect exactly what failed.

The platform should demonstrate that it can detect problems that a basic uptime monitor would miss.

12. Key Product Metric

The most important metric should eventually be:

Meaningful failures detected

Not:

Number of API checks performed.

We want to measure whether the platform catches problems that actually matter.